

SMART RESUME ANALYZER WITH AI-BASED FEEDBACK

Deterministic ATS Screening Engine, Multi-Factor Scoring & Actionable Optimization Feedback

Author / Candidate: subbaraju-cmd | Stack: Python Flask, Unicorn & Render Cloud | Academic Capstone 2026

Core Project Objective

- **Applicant-ATS Gap:** Over 75% of qualified resumes are filtered out prematurely by automated Applicant Tracking Systems.
- **Instant Transparency:** Provides job seekers with real-time, deterministic ATS compatibility screening before applying.
- **Zero Token Bleed:** Eliminates false-positive substring matches on technical keywords like C++, Node.js, and CI/CD.
- **Client-Side PDF:** Exports print-ready vector PDF audit reports directly in the browser with zero server overhead.

Technical Innovations

- **Lookaround Regex:** Custom token boundary regex `(?<![a-zA-Z0-9])kw(?:![a-zA-Z0-9])` isolating programming terms.
- **Relevance Guardrails:** Mathematical caps ensuring 0% technical match resumes never receive a passing score.
- **Google X-Y-Z Formula:** Dynamic generation of high-impact achievement bullet points for missing role skills.
- **Modular Frontend:** Decoupled 10-component view architecture in templates/components/ for easy maintenance.

Live Production Links

- **GitHub Repository:**
<https://github.com/subbaraju-cmd/Smart-Resume-Analyzer>
- **Live Render URL:** <https://smart-resume-analyzer-utj7.onrender.com>
- **Production Stack:** Python 3.14, Flask 3.1, Unicorn 23.0 WSGI, pypdf 6.19, python-docx 1.2.
- **DevOps & CI/CD:** Render Blueprint (render.yaml) with automatic deployment on git push to main branch.

The Recruitment Bottleneck & Our Engineering Approach

The Problem: Why Traditional Screeners Fail

- **Opaque Automated Filtering:** Over 75% of resumes are filtered out before reaching a human recruiter due to missing exact keywords, non-standard section headers, or unparseable layout formatting.
- **Artificial Score Inflation Flaw:** Traditional online resume checkers naively award 65-75 points out of 100 simply for having an email address, telephone number, and generic headings, even when the candidate has 0% technical competence.
- **Regex Word Boundary Traps:** Standard regex `\b` tokenizers fail completely on engineering terms with symbols: `\bC++\b` fails because `'+'` is treated as a word boundary, while `'Java'` falsely matches `'JavaScript'`.
- **Superficial Black-Box Feedback:** Existing checkers display an arbitrary percentage score without telling the candidate which exact keywords are missing or how to formulate strong achievement statements.

Our Solution: Deterministic ATS Engine

- **Deterministic & Cost-Effective:** Zero reliance on unpredictable, hallucination-prone, and expensive LLM API calls. Delivers completely reproducible, sub-second execution speed on standard web servers.
- **Dual-Format Extraction Pipeline:** Robust text extraction supporting both PDF (via `pypdf` stream buffer) and DOCX (via `python-docx` paragraph/table XML parser) documents with UTF-8 normalization.
- **Guarded Multi-Factor Scoring:** Mathematical model combining role match percentage, technical depth, section taxonomy, and contact info, bounded by strict anti-inflation relevance guardrails.
- **Comprehensive Section Auditing:** Audits 5 essential sections (Contact, Summary, Education, Experience, Skills) and validates RFC-compliant email and telephone patterns.
- **Actionable Optimization Roadmap:** Provides concrete phrasing substitutions and applies Google's X-Y-Z bullet formula for missing role competencies.

Core Technical Innovations: Regex Engine & Guardrail Validation

1. Lookaround Boundary Regex Engine

Solving Programming Language Token Bleed:

`pattern = rf"(?<![a-zA-Z0-9]){re.escape(keyword)}(?![a-zA-Z0-9])"`

- Eliminates false substring matches: 'Java' never matches inside 'JavaScript'; 'C' never matches 'CSS' or 'CI/CD'.
- Accurately captures punctuated engineering tokens: C++, C#, .NET, and Node.js.

2. Anti-Inflation Scoring Guardrails

Mathematical Model & Guardrail Formula:

$$\text{Score} = (\text{RoleMatch} \times 0.40) + (\text{TechDepth} \times 0.25) + (\text{SectionArch} \times 0.25) + (\text{Contact} \times 0.10)$$

- Hard Cap 1: If Role Match = 0% → Final score is hard-capped at ≤ 25 / 100 (Unqualified resumes never pass).
- Hard Cap 2 & 3: Match < 25% capped at ≤ 40/100; Match < 50% capped at ≤ 55/100. Passing scores require real skills.

Candidate Test Profile	Target Job Role	Naive ATS Score	Our Engine Score	Validation Outcome & Recruiter Impact
Personal Trainer Resume	Web Developer	65 / 100 (False Pass)	24 / 100 (Guarded)	Relevance guardrail correctly rejects candidate with 0% role match.
Junior Python Developer	Python Developer	68 / 100 (Uninformative)	68 / 100 (Actionable)	Identifies 57.1% role match; highlights missing Docker & Celery skills.
Senior AI Engineer	AI Engineer	88 / 100 (No Syntax Check)	91 / 100 (Verified)	High match validates PyTorch, TensorFlow, NLP, and impact action verbs.
Scanned / Image Resume	Data Analyst	40 / 100 (Silent Fail)	Explicit Red Alert	Catches zero-text extraction and returns actionable on-screen warning.

System Architecture & Modular File Mapping

The application follows a strictly decoupled Model-View-Controller (MVC) architecture with isolated core/ processing modules and 10 standalone templates/components/ views for feature-friendly maintainability:

Module / File Path	Architectural Layer	Key Technical Responsibility & Logic	Underlying Technology
core/parser.py	Extraction Layer	Multi-format document parsing extracting clean UTF-8 text streams from PDF and DOCX files without third-party API dependencies.	pypdf, python-docx
core/section_analyzer.py	Audit Layer	Validates RFC-compliant email syntax, telephone numbers, LinkedIn/GitHub URLs, and audits 5 essential resume section headers.	Python re, String Sanitizer
core/ats_scorer.py	Scoring Layer	Executes lookaround regex matching across 50+ master skills, computes role match %, audits impact verbs, and enforces guardrails.	Lookaround Regex, JSON Taxonomy
core/feedback_generator.py	Recommendation Layer	Generates Google X-Y-Z bullet points, weak vs. strong phrasing transformations, and tailored suggestions for missing competencies.	Rule-Based Heuristics
templates/components/	Presentation Layer	10 isolated HTML component views (metrics, keyword badges, section checks, Google X-Y-Z, phrasing comparison, PDF export button).	Jinja2, HTML5, CSS3 Grid
app.py & Procfile	Gateway & DevOps	Flask web endpoint, route handling (/ and /upload), input validation, secure_filename sanitization, and Gunicorn WSGI startup.	Flask 3.1, Gunicorn 23.0

Actionable Feedback Engine & Phrasing Transformation

THE GOOGLE X-Y-Z BULLET POINT FORMULA

"Accomplished [X], as measured by [Y], by doing [Z]" — Replaces passive job descriptions with high-impact, quantified achievement statements.

Weak / Passive Phrasing (Before)	Identified ATS Flaw	High-Impact Transformation (After - Google X-Y-Z)
Worked on web development and fixed bugs in the system.	Passive verb ('worked'), no measurable metrics, tech stack omitted.	Engineered responsive full-stack features using React and Flask, resolving 45+ critical bugs and improving system uptime by 22%.
Responsible for data analysis and preparing reports.	Duty-oriented description, no scale or business impact indicated.	Automated daily ETL pipelines using Python and Pandas, analyzing 500K+ records and reducing reporting turnaround from 6 hours to 45 mins.
Assisted team in deploying machine learning models.	Minimizes contribution ('assisted'), zero deployment specifications.	Architected and containerized PyTorch NLP classification models with Docker, boosting inference throughput by 3.5x in production.
Helped migrate company databases to the cloud.	Vague action verb ('helped'), missing architecture and efficiency data.	Orchestrated seamless PostgreSQL migration to AWS RDS, achieving 99.99% availability and cutting annual infrastructure costs by \$18,000.

Production Deployment, Specifications & Future Roadmap

Live Production Hosting

Deployed & Verified Live on Render Cloud:

- Live URL: <https://smart-resume-analyzer-utj7.onrender.com>
- Continuous Deployment: Automated build triggered on git push to main.
- Health Status: HTTP 200 OK verified across upload & analysis endpoints.

Future Technical Roadmap

Planned Engineering Enhancements:

- OCR Engine Integration: Tesseract OCR for scanned image PDFs and multi-column visual resumes.
- Semantic Embeddings: Lightweight on-device MiniLM embeddings for contextual synonym matching.
- Job Description Auditing: Real-time parsing of custom recruiter job descriptions.

Production Metric / Component	Live Configuration & Value	Operational Significance & Quality Guarantee
Hosting Platform & Region	Render Cloud Web Service (Oregon)	24/7 cloud container with automatic TLS certificates and Cloudflare CDN.
WSGI Production Server	Gunicorn 23.0.0 (web: gunicorn app:app)	Multi-worker process management with dynamic \$PORT binding on Linux.
Version Control & CI/CD	GitHub subbaraju-cmd/Smart-Resume-Analyzer	Automated deployment webhook on git push to main branch with render.yaml.
Vector PDF Report Export	html2pdf.js + static/css/print.css	Client-side vector rendering; zero server overhead or memory leaks.